

23.To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

```
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipe1[2];
    int pipe2[2];

    pipe(pipe1);
    pipe(pipe2);

    pid_t p1 = fork();

    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);

        perror("grep");
        exit(1);
    }

    pid_t p3 = fork();

    if (p3 == 0) {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("wc");
        exit(1);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("Pipeline completed successfully.\n");

    return 0;
}
```

```
bindu@Bindusree: ~  
bindu@Bindusree:~$ nano skill23.c  
bindu@Bindusree:~$ gcc skill23.c -o skill23  
bindu@Bindusree:~$ ./skill23  
78  
Pipeline completed successfully.  
bindu@Bindusree:~$
```